Task 2. Ones

Dado un arreglo oculto de N bits – $a_0, \ldots, a_{N-1}$. En una consulta, puedes elegir un subconjunto de posiciones en la secuencia y flipear sus bits en esas posiciones. Flipear un bit 0 lo cambia a un bit 1 y flipear un bit 1 lo cambia a un bit 0. Luego de cada consulta, se te da la longitud de la subarreglo más largo de 1s consecutivos en el nuevo arreglo. **Las consultas persisten, es decir un bit flipeado en una consulta anterior mantendra su nuevo valor hasta que vuelva a ser parte de una nueva consulta.**

Se desea encontrar donde se encontrara el subarreglo más largo de ones consecutivos(o alguno de estos en caso de que existan varios). luego de todas las consultas. Escriba un programa que realice esto con la menor cantidad de queries posibles.

Implementation details

There are T subtests per test.

Your program needs to implement a function with the following prototype:

```
std::pair<int,int> find_longest_subarray_of_ones(int n);
```

It will be called T times per test and receives the integer N as a parameter – the length of the hidden array. The function should return a `pair{l,r}`, where `l` is the index of the leftmost part of the subarray and `r` is the index of the rightmost part of the subarray.

You should use the function `flip_bits` to make queries:

```
int flip_bits(const std::vector<bool> &flips);
```

It receives a vector of N bits $flips_0, \ldots, flips_{n-1}$, where $flips_i = 1$ would signify that a_i should be flipped. After flipping the required bits, the function returns the length of the longest subarray of ones in the hidden sequence.

You must submit the file `ones.cpp` to the system which contains the `find_longest_subarray_of_ones` function. It may contain other code and functions necessary for your work, but it must not contain the `main` function. Also, you must not read from the standard input or print to the standard output. Your program must also include the `ones.h` header file by instruction to the preprocessor:

```
#include "ones.h"
```

Local testing

You are given the file Lgrader.cpp, which can be compiled with your program to test it. It will read the number of subtests T for the test, followed by a description of each of them. For each subtest, first the integer N will be given, followed by $a_0 \ldots a_{N-1}$ on the next line. If it runs correctly, the program will output 1 and the maximum number of queries you have requested. Otherwise, it will print 0.

Constraints

$T = 5$

$1 \leq N \leq 10^4$

$0 \leq a_i \leq 1$

Scoring

If your program is wrong for any subtest, your score will be 0. Otherwise, the score you receive for the problem depends on the maximum number of queries Q your program has used to solve a single subtest. The score that you receive for the problem is :

$$\left(\frac{85}{max\left(85, Q\right)}\right)^{0.294} \times 100$$

Ejemplo de comunicación

Digamos que el arreglo inicial es 1, 0, 1. Entonces una forma de mostrar la comunicación sería:

Función del Contestant	Programa del juez
	Llamar **find_longest_subarray_of_ones(3)**
Llamar **flip_bits({0, 0, 0})**	Retornar el valor **1**
Llamar **flip_bits({0, 1, 0})**	Retornar el valor **3**
Retornar el valor **{0, 2}**	